

LAB 2-prep : Hadoop: Environment Setup

Complete this before the Hadoop lab session: it walks you through installing and starting a pseudo-distributed Hadoop cluster (HDFS + YARN) on your own machine, and verifying your Maven environment is ready to build Java MapReduce programs against it.

Exercise 1. Setting Up a Pseudo-Distributed Cluster

Hadoop is a distributed platform written in Java, made of two main services:

- **HDFS** (Hadoop Distributed File System): a distributed file system suited to large volumes of data.
- **YARN** (Yet Another Resource Negotiator): a cluster resource manager based on containers. Yarn applications (Hadoop MapReduce, Spark, ...) run their tasks inside these containers.

HDFS and YARN are independent of each other and each follows a master/slave layout: for HDFS the master is the **NameNode** and the slaves are **DataNodes**; for YARN the master is the **ResourceManager** and the slaves are **NodeManagers**. Hadoop MapReduce, the application we use in this lab, is itself just a YARN application that runs MapReduce computations on top of YARN and HDFS.

Hadoop has three modes of operation:

- **local mode**: a single process simulates the whole platform (no HDFS, the local filesystem is used instead), handy to develop and debug before running on a real deployment.
- **pseudo-distributed mode**: one instance of every Hadoop daemon runs on the local machine, communicating over loopback: this is what we use in this lab.
- **distributed mode**: Hadoop runs on a real distributed system (cluster, grid, cloud): this is the production mode.

Question 1. Download the latest stable release of Hadoop from the Apache Hadoop releases page and extract `hadoop-3.x.y.tar.gz` into your home directory. Hadoop needs the `JAVA_HOME` environment variable to point at a JVM. Java 17 can break YARN's startup, so make sure you are on **Java 11** (`java --version`). Add to your `.bashrc`:

```
export JAVA_HOME=<path to your JVM>
export MY_HADOOP_HOME=<your Hadoop directory>
export PATH=$PATH:$MY_HADOOP_HOME/bin
export PATH=$PATH:$MY_HADOOP_HOME/sbin
export HADOOP_HOME=$MY_HADOOP_HOME
```

then source `~/ .bashrc` and check `hadoop version` prints something sensible.

Question 2. The executables in `$HADOOP_HOME/bin` are mostly shell scripts: `hdfs` manipulates the distributed file system, `yarn` manipulates the resource manager, `mapred` administers Hadoop MapReduce jobs (the old unified `hadoop` command is deprecated). The `$HADOOP_HOME/sbin` scripts `start-dfs.sh`, `stop-dfs.sh`, `start-yarn.sh`, `stop-yarn.sh` wrap these to deploy/tear down the whole platform: you will need exactly those four.

Question 3. Before starting Hadoop, edit its configuration files under `$HADOOP_HOME/etc/hadoop`. We only use pseudo-distributed mode (every daemon on `localhost`):

- `etc/hadoop/workers` lists the worker machines: just `localhost`.
- `etc/hadoop/hdfs-site.xml`: HDFS configuration; `dfs.replication = 1` disables replication (default is 3), see `data/conf/hdfs-site.xml` in week 2's main lab resources.
- `etc/hadoop/core-site.xml`: the NameNode's URL, see `data/conf/core-site.xml`. Check port 9000 is free first (`netstat -a | grep tcp | grep LISTEN | grep 9000` should print nothing).
- `etc/hadoop/mapred-site.xml` (create it if missing): tells Hadoop MapReduce to run on YARN, see `data/conf/mapred-site.xml`.
- `etc/hadoop/yarn-site.xml`: YARN's shuffle auxiliary service, see `data/conf/yarn-site.xml`.

Double-check every XML file has exactly one `<?xml ... ?>` and one `<?xml-stylesheet ... ?>` declaration before continuing.

Question 4. Start the platform:

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
```

Wait a few seconds, then use `ps uxf` to see which daemons were launched.

Question 5. Each daemon also starts a web server for monitoring. Run `netstat -na | grep tcp | grep LISTEN` and, restricting yourself to ports above 1000, work out which service each port belongs to.

Exercise 2. Verifying Your Maven Environment

All Java code for the Hadoop lab is a Maven project: no particular IDE is required, use whatever editor or IDE you like, as long as you have a recent Maven (`mvn --version`) and a Java 11 JDK on your `PATH`.

Question 1. Fetch the Hadoop lab's starter project (`week2/lab/resources_maven/` in the course repository) and build it:

```
cd resources_maven
mvn compile
```

This downloads the Hadoop client libraries declared in `pom.xml` and compiles the given `WordCount` and `SequenceFileCreator` programs. A successful `BUILD SUCCESS` means your environment is ready: the full end-to-end run (uploading data, packaging a jar, submitting it to YARN) happens as the first exercise of the main lab.